

Hierarchical Reduce: Depth vs Fan-in, by Hand

When one reduce call can't hold all the partials — how deep the tree goes, how many calls it costs, and the fan-in that minimises both.

What this document does. It takes the one scaling problem from Module 2.2's "two-level map-reduce" section — you have produced M map partials and need to combine them, but they will not all fit into a single reduce prompt — and works the geometry of the reduce *tree* by hand. We fix $M = 64$ partials of $s = 400$ tokens each, overhead $B = 500$, window $W = 8000$, and compare three fan-ins: $f = 8$ (a shallow tree), $f = 2$ (a deep binary tree), and the flat $f = 64$ (one giant reduce). For each we count the tree depth, the number of reduce calls, the largest reduce input, and whether it overflows. Every figure is reproduced by the Python program in Appendix A.

Where this sits. This is **Topic 2 of Module 2.2**. It picks up exactly where Topic 1 ends: Topic 1 showed Map-Reduce's reduce step overflows once $D > (W - B)/s$; this document is what you do next. Its companions: Module 2.2 Topic 1 (Stuff vs Map-Reduce vs Refine — the strategy that generates these M partials) and Module 1.0 (async concurrency gates — the semaphores that run each tree level in parallel).

Concepts covered, in order: the reduce tree as a f -ary tree over M leaves · depth = $\lceil \log_f M \rceil$ · call count = $\lceil (M-1)/(f-1) \rceil$ · latency = depth × reduce-latency (parallel within a level, serial across levels) · the flat-reduce overflow $B + M s > W$ · the optimal fan-in $f^* = \lfloor (W-B)/s \rfloor$ that bounds reduce input while minimising depth.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: 64 partials that won't fit in one reduce

In a map-reduce job, the map step has already run: you hold M small partials, each a typed summary of one input. The reduce step must fold them into one answer. The naive move is a single reduce call over all M partials — but if M is large, that one prompt blows the window. The module's fix is *hierarchical reduce*: group the partials, reduce each group, then reduce the group summaries, recursing until one root remains. The question this document answers is purely geometric — **how do you group?** A reduce that merges f summaries per call (the *fan-in*) defines an f -ary tree over the M partials, and f alone fixes the tree's depth, its call count, and whether any node overflows.

The workload. A research agent has fanned a topic into many sub-questions and produced $M = 64$ partial answers, each a $s = 400$ -token structured summary. Every reduce call carries $B = 500$ tokens of overhead and the model window is $W = 8000$ tokens.

Quantity	Symbol	Value here
Partials to combine	M	64
Tokens per partial (summary)	s	400 tokens
Overhead per reduce call	B	500 tokens
Reduce fan-in (summaries merged per call)	f	varied: 2, 8, 18, 64
Context window	W	8,000 tokens
Latency of one reduce call	L_r	2 s (illustrative)

Notation. The *depth* of the tree is the number of reduce *levels* from leaves to root. *Calls* is the total number of reduce calls across all levels. The *reduce input* of a single call that merges f partials is $B + f s$ tokens. “Overflow” means $B + f s > W$.

Intuition

A reduce tree is a knockout tournament for summaries. Fan-in f is how many summaries play each match. Small f (say 2) means tiny matches but many rounds — a deep bracket. Large f (say 64) means one enormous match that decides everything in a single round — but the room may not hold 64 players at once. The art is picking the largest f whose match still fits in the room, so the tournament ends in as few rounds as possible.

2 Step 1 — depth: how many levels to reach the root

Each reduce level shrinks the number of live items by a factor of f : start with M partials, after one level you have $\lceil M/f \rceil$ group summaries, after two you have $\lceil M/f^2 \rceil$, and so on until one root remains. The number of levels needed is the smallest L with $f^L \geq M$, i.e.

$$\text{depth}(M, f) = \lceil \log_f M \rceil. \quad (1)$$

For $M = 64$: with $f = 8$, $\log_8 64 = 2$ exactly, so $\text{depth} = 2$ ($64 \rightarrow 8 \rightarrow 1$). With $f = 2$, $\log_2 64 = 6$, so $\text{depth} = 6$ ($64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$). With the flat $f = 64$, $\log_{64} 64 = 1$, $\text{depth} = 1$ — a single reduce. Depth falls as f rises; that is the whole appeal of a wide fan-in.

Mini-example — this operation in isolation

Watch the live-count collapse for $f = 8$: level 0 holds 64 partials; one level of group-reduces ($64/8 = 8$ calls) leaves 8 group summaries; a second level ($8/8 = 1$ call) leaves 1 root. Two levels, done — because $8^2 = 64$ lands exactly on M . Whenever M is a clean power of f , the depth is the exponent and there is no rounding; $64 = 8^2 = 2^6 = 64^1$, giving depths 2, 6, and 1 for $f = 8, 2, 64$ respectively.

3 Step 2 — call count: how many reduces in total

Depth is the number of *levels*; it is not the number of *calls*. Each reduce call merges f items into 1, so it removes $f - 1$ items from the live set. The tree starts with M items and must end with 1, a net removal of $M - 1$ items. Therefore the total number of reduce calls is

$$\boxed{\text{calls}(M, f) = \left\lceil \frac{M-1}{f-1} \right\rceil} \quad (2)$$

This counts every call at every level at once — elegant, because “items removed” is conserved regardless of how the tree is shaped. Check the three cases for $M = 64$:

- $f = 8$: $\lceil 63/7 \rceil = \lceil 9 \rceil = 9$ calls (8 at level 1, 1 at level 2). ✓
- $f = 2$: $\lceil 63/1 \rceil = 63$ calls (a binary tree on 64 leaves has 63 internal nodes). ✓
- $f = 64$: $\lceil 63/63 \rceil = 1$ call. ✓

So fan-in trades calls against depth in opposite directions: $f = 2$ gives the *most* calls (63) but maximal depth, while $f = 64$ gives the *fewest* (1) at minimal depth. More fan-in is fewer calls *and* fewer levels — it looks strictly better, until we add the window.

Intuition

Equation (2) is just conservation of work. Every partial except the final survivor must be “consumed” by exactly one reduce call, and each call consumes $f - 1$ of them (it keeps one result alive). So the call count barely depends on the tree’s shape — only on how many items each call swallows. This is why doubling the fan-in roughly halves the number of reduce calls.

4 Step 3 — latency: depth sets the wall clock

Within one level the reduce calls are independent — group A’s reduce does not need group B’s result — so they run *in parallel* (gated by the Module 1.0 semaphore). Across levels they are serial: level 2 cannot start until level 1’s summaries exist. So wall-clock latency is governed by *depth*, not by the call count:

$$\text{latency}(M, f) = \text{depth}(M, f) \times L_r = \lceil \log_f M \rceil L_r. \quad (3)$$

With $L_r = 2$ s: the flat $f = 64$ tree finishes in $1 \times 2 = 2$ s (one call, one level); $f = 8$ takes $2 \times 2 = 4$ s; the binary $f = 2$ tree takes $6 \times 2 = 12$ s even though most of its 63 calls overlap. **The 63 calls of the binary tree are cheap in tokens but expensive in time**, because its six serial levels cannot be collapsed. This is the exact dual of Topic 1’s Refine penalty: serial structure, not call volume, is what the clock punishes.

5 Step 4 — the catch: a wide fan-in can overflow the window

If wide fan-in means fewer calls, fewer levels, and lower latency, why not always set $f = M$ and do a single flat reduce? Because a call merging f partials must *fit* all of them in one prompt: its input is $B + f s$, and it runs only if $B + f s \leq W$. The flat $f = 64$ reduce needs

$$B + 64 s = 500 + 64 \cdot 400 = 500 + 25,600 = 26,100 \text{ tokens} > 8,000 = W.$$

It **overflows by more than 3×** — the very failure Topic 1 predicted at $D > (W - B)/s$. That is why the tree exists at all: deepening it ($f < M$) bounds every node’s input to $B + f s$ while adding levels. Here is the full trade-off for $M = 64$:

Fan-in f	Depth	Calls	Reduce input $B+fs$	Fits?	Latency
2	6	63	1,300	yes	12 s
8	2	9	3,700	yes	4 s
18	2	4	7,700	yes (tightest fit)	4 s
64	1	1	26,100	no ($> W$)	—

Watch out

The flat reduce is a trap precisely because it is the cheapest-looking row: 1 call, 1 level, 2 s. But $B + Ms$ scales with the *whole* batch, so it is the row most likely to overflow — and when it does, it does not degrade, it simply cannot run. Never size a reduce by call count alone; size it by its single-call input $B + fs$ against W first, and only then minimise depth.

6 Step 5 — the optimal fan-in

The table makes the rule obvious. Latency depends only on depth, and depth $\lceil \log_f M \rceil$ *decreases* as f grows — so to minimise latency we want f as *large* as possible. The only thing stopping us is the window: a reduce call must satisfy $B + fs \leq W$. So take the largest fan-in that still fits:

$$f^* = \left\lfloor \frac{W - B}{s} \right\rfloor \quad (4)$$

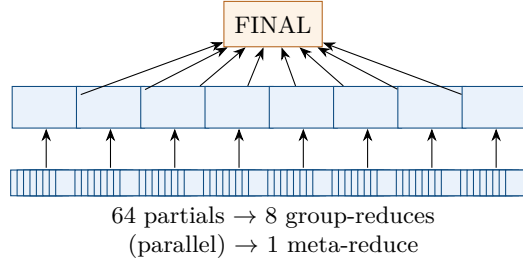
With our numbers $f^* = \lfloor (8000 - 500)/400 \rfloor = \lfloor 18.75 \rfloor = 18$. Check the boundary: at $f = 18$ the reduce input is $500 + 18 \cdot 400 = 7,700 \leq 8000$ — fits. ✓ At $f = 19$ it is $500 + 19 \cdot 400 = 8,100 > 8000$ — overflows. So $f^* = 18$ is the widest legal fan-in. Its tree has depth $\lceil \log_{18} 64 \rceil = 2$ and $\lceil 63/17 \rceil = \lceil 3.7 \rceil = 4$ calls — it matches the $f = 8$ tree's minimal depth of 2 (and thus its 4 s latency) while using *fewer* reduce calls (4 vs 9), because each call does more work.

Mini-example — this operation in isolation

Why does the widest-that-fits win? For $M = 64$ and any f from 9 to 64, the depth is already at its floor for a two-level tree ($f^2 \geq 64 \iff f \geq 8$), so latency is pinned at 2 levels. Among those, larger f means each level has fewer, fatter calls — $f = 18$ needs only 4 calls total versus $f = 8$'s 9. Push to $f = 19$ and you save nothing on depth but break the window. So $f^* = 18$ sits at the exact corner: maximal fan-in, minimal depth, no overflow. That is the sweet spot Equation (4) names.

7 The reduce tree, drawn

$f = 8$: depth 2, 9 calls, input 3700



Level 0 is the 64 leaf partials; eight parallel group-reduces (fan-in 8) collapse them to eight group summaries; one meta-reduce folds those into the root. Two levels deep, nine calls total — and no single call ever ingests more than $B + 8s = 3,700$ tokens. The drawing *is* Equations (1)–(2).

8 Hierarchical-reduce cheat-sheet

Tree depth (levels)	$\text{depth} = \lceil \log_f M \rceil$
Total reduce calls	$\text{calls} = \lceil (M - 1) / (f - 1) \rceil$
Single reduce-call input	$B + f s$
Latency (parallel/level, serial/levels)	$\text{depth} \times L_r = \lceil \log_f M \rceil L_r$
Flat reduce ($f = M$) overflows when	$B + M s > W$
Largest safe fan-in	$f^* = \lfloor (W - B) / s \rfloor$
Levels handled by depth ℓ	$M \leq f^\ell$ (module: 2 levels $\approx k^2$)

9 An honest word about these numbers

The sizes here ($M = 64$, $s = 400$, $B = 500$, $W = 8000$, $L_r = 2$ s) are illustrative round numbers, chosen so the trees land on clean powers ($64 = 8^2 = 2^6$) and the optimal fan-in falls at a tidy $f^* = 18$. Real values shift the constants: a 128k-token window makes f^* hundreds wide, so most realistic batches collapse to a single flat reduce or a two-level tree — the module’s note that “2 levels handles N up to $\approx k^2$.” Larger partials s shrink f^* and force deeper trees. What is *not* illustrative is the structure: depth is $\lceil \log_f M \rceil$, calls are $\lceil (M-1)/(f-1) \rceil$, latency tracks depth because levels serialise while a level parallelises, and the only constraint on fan-in is $B + f s \leq W$. Change the constants and the depths and call counts move; the logarithm-versus-window trade-off does not. The single rule worth memorising is Equation (4): make each reduce as wide as the window allows, then let depth fall out.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the depth / calls / overflow table and the optimal-fan-in derivation; change **M**, **s**, **B**, or **W** to size your own reduce tree.

```

1  import math
2
3  # Topic 2 -- Hierarchical reduce: depth vs fan-in, by hand.
4  M, s, B, W = 64, 400, 500, 8000 # items, summary tokens, overhead, window
5  L = 2.0 # seconds per reduce call (illustrative)
6
7  def depth(M, f): return math.ceil(math.log(M, f))
8  def calls(M, f): return math.ceil((M - 1) / (f - 1))
9  def reduce_input(f): return B + f * s
10
11 print("fan-in f depth calls reduce_in overflow latency(s)")
12 for f in (2, 8, 18, 64):
13     dpth = depth(M, f)
14     c = calls(M, f)
15     ri = reduce_input(f)
16     print(f" f={f:2d} {dpth:3d} {c:3d} {ri:7d} {str(ri > W):8} {dpth * L:5.1f}")
17
18 # Largest fan-in that keeps a single reduce input below the window:
19 fmax = (W - B) // s
20 print(f"\nLargest safe fan-in f_max = floor((W-B)/s) = {fmax}")
21 print(f" reduce_in(f={fmax}) = {reduce_input(fmax):5d} (< {W}? {reduce_input(fmax) < W})")
22 print(f" reduce_in(f={fmax+1}) = {reduce_input(fmax+1):5d} (< {W}? {reduce_input(fmax+1) < W})")
23
24 print(f" depth(f={fmax}) = {depth(M, fmax)}, calls = {calls(M, fmax)}")
25 print(f"\nOptimal f for M={M}: f*={fmax} -> depth {depth(M, fmax)}, "
26       f"{calls(M, fmax)} calls, reduce_in {reduce_input(fmax)} < {W}")
27
28 # Expected output:
29 # fan-in f depth calls reduce_in overflow latency(s)
30 # f= 2 6 63 1300 False 12.0
31 # f= 8 2 9 3700 False 4.0
32 # f=18 2 4 7700 False 4.0
33 # f=64 1 1 26100 True 2.0
34
35 # Largest safe fan-in f_max = floor((W-B)/s) = 18
36 # reduce_in(f=18) = 7700 (< 8000? True)
37 # reduce_in(f=19) = 8100 (< 8000? False)
38 # depth(f=18) = 2, calls = 4
39
40 # Optimal f for M=64: f*=18 -> depth 2, 4 calls, reduce_in 7700 < 8000

```

— end of document —